SELECTED WORKS / 웹·백엔드

Synapse

학습 플랫폼을 지탱하는 인증·결제·알림·감사 서비스

노트와 반복 학습을 연결하는 팀 프로젝트에서 플랫폼 서비스의 인증, 결제, 알림, 감사 영역을 담당했습니다.

https://github.com/MaoEmong/synapse-team-archive

웹 상세 소개 보기

Java 21 · Spring Boot · Spring Modulith · PostgreSQL · Redis · Kafka · Avro

프로젝트 성격	학습 플랫폼 팀 아카이브
담당 영역	인증 · 결제 · 알림 · 감사
서비스 내부 구조	Spring Modulith 모듈 경계
서비스 간 계약	Kafka · Avro · Schema Registry
공개 상태	백엔드 구현 및 테스트 / Flutter mock 프로토타입

01 / CONTEXT

어떤 문제에서 시작했나요?

노트를 학습카드와 복습으로 연결하려면 지식·학습 기능 외에도 공통 계정, 요금제, 알림, 운영 기록이 필요합니다. 각 서비스가 계정 상태와 권한을 제각각 판단하면 사용자 정지나 구독 변경이 일관되게 반영되기 어렵습니다. platform-svc는 다른 서비스가 신뢰할 인증 기준과 공통 플랫폼 기능을 제공하는 역할을 맡습니다.

정상 요청 이후의 상태 관리도 중요했습니다. 로그인은 여러 기기의 세션을 관리해야 하고, 결제 웹훅과 Kafka 알림은 중복 수신될 수 있으며 외부 전송은 실패할 수 있습니다. 세션과 이벤트의 상태를 보존하고 재처리 기준을 나누어, 같은 요청이 반복되거나 외부 시스템이 실패할 때의 처리 흐름을 구성했습니다.

02 / PRODUCT FLOW

기능과 사용자 흐름

계정과 세션

이메일·OAuth 인증, RS256 JWT, 갱신 토큰 회전과 다중 기기 세션을 관리합니다.

구독과 결제

Stripe Checkout·서명 검증 웹훅·구독·결제 이력과 영수증 조회를 구성합니다.

알림과 감사

Kafka 알림 요청을 채널별 상태로 처리하고 도메인 이벤트를 감사 기록으로 적재합니다.

한 번의 사용을 따라가 보면

01 계정 생성과 로그인

사용자가 가입하고 로그인하면 access token과 refresh cookie를 받습니다. RS256 JWT를 공통 인증 기준으로 사용하며 OAuth, MFA, 비밀번호 재설정과 같은 계정 흐름을 플랫폼에 모았습니다.

02 세션 갱신과 계정 관리

갱신 요청은 기존 토큰을 새 토큰으로 회전합니다. 사용자별 잠금 아래 활성 세션을 최대 5개로 관리하고, 오래된 세션을 제거합니다. 사용자 정보와 OAuth 연결, 테넌트 관리 API도 같은 계정 맥락에서 제공합니다.

03 Checkout에서 구독 반영까지

사용자가 요금제를 선택하면 Checkout 세션을 생성합니다. 이후 서명이 유효한 Stripe 이벤트를 받아 구독을 활성화하거나 해지하고, 결제 이력과 영수증을 테넌트 범위로 조회합니다.

04 알림과 감사 기록

NotificationSend 이벤트를 채널별로 처리하고 이미 보낸 알림은 생략합니다. 학습·노트·참여 이벤트는 감사 기록으로 모아 관리자가 조회할 수 있도록 구성합니다. 이는 사용자 기능과 운영 추적을 연결하는 흐름입니다.

03 / ARCHITECTURE

구조와 책임을 나눈 방식

01 팀 전체의 서비스 경계

지식, 학습, 참여, 플랫폼 서비스를 나누고 Kafka 이벤트로 연결합니다. platform-svc는 인증·사용자·결제·알림·감사·관리자 모듈을 하나의 Spring Boot 애플리케이션 안에서 관리합니다. 팀원들은 인프라와 AI 학습카드 등 각 영역을 맡아 개발했고, 담당한 platform-svc는 공통 플랫폼 기능을 제공합니다.

02 영속 데이터와 세션 캐시

PostgreSQL과 Flyway로 플랫폼 데이터를 관리하고 Redis를 갱신 토큰 조회에 사용합니다. RefreshTokenService는 토큰 해시를 저장하며 캐시 미스 시 DB의 만료 조건을 확인합니다. DB 커밋 뒤 Redis를 갱신해 두 저장소의 변경 시점을 조정합니다.

03 타입이 있는 이벤트 계약

Kafka 메시지는 Avro record와 eventId를 사용합니다. 가입 이벤트 발행과 외부 도메인 이벤트의 감사 로그 적재, 알림 요청 소비를 분리합니다. Kafka는 기본 비활성화이며 환경 설정으로 producer·consumer·outbox publisher를 켜는 구조입니다.

04 외부 제공자 어댑터

결제는 Stripe Checkout과 서명 검증 웹훅, 알림은 FCM·SES를 연결합니다. 제공자 설정이 없는 알림 채널은 건너뛰고 발송 이력과 실패 상태를 관리합니다. 실제 계정·네트워크까지 포함한 동작은 별도의 실행 환경이 필요합니다.

04 / ENGINEERING CASES

구현하면서 선택한 것

CASE 01 / 갱신 토큰의 DB·Redis 변경 순서

마주한 문제

여러 기기에서 로그인과 토큰 회전이 겹치면 세션 수 제한과 기존 토큰 제거가 서로 충돌할 수 있습니다.

선택과 구현

사용자별 세션 잠금 안에서 저장·회전·삭제를 처리하고 DB 트랜잭션 커밋 뒤 Redis 변경을 실행합니다. Redis에 없으면 DB의 해시와 만료 시각을 확인합니다.

남은 비용과 한계

DB 커밋 이후 Redis 갱신이 실패하면 두 저장소의 상태가 어긋날 수 있습니다. 캐시 장애와 갱신 실패에 대한 정합성·복구 정책을 별도로 점검해야 합니다.

RefreshTokenService 구현

CASE 02 / 웹훅의 서명 검증과 중복 방지

마주한 문제

결제 제공자의 재전송을 그대로 적용하면 같은 이벤트로 구독·결제 상태가 반복 변경될 수 있습니다.

선택과 구현

원본 payload의 Stripe 서명을 먼저 검증하고, processed event를 insertIfAbsent로 기록한 뒤 처음 처리하는 이벤트만 유형별 로직으로 보냅니다. 조회는 사용자에서 얻은 tenantId로 범위를 제한합니다.

남은 비용과 한계

테스트에는 실제 서명 검증을 포함했습니다. Stripe 운영 환경의 실결제·장애·이벤트 순서 역전은 별도 시나리오로 검증해야 합니다.

BillingService 구현

CASE 03 / 알림을 채널별 상태로 관리

마주한 문제

알림 이벤트가 재수신되거나 한 채널 발송이 실패해도 이미 보낸 채널을 불필요하게 다시 보내지 않아야 합니다.

선택과 구현

eventId와 channel로 기존 알림을 찾고 sent 상태면 생략합니다. 시도 횟수와 실패 원인을 저장하고 예외를 다시 전달하며, 미설정 채널과 일반 이메일의 일일 한도도 구분합니다.

남은 비용과 한계

외부 발송 직후 DB 상태 저장 전 장애가 나면 재발송 가능성이 남습니다. DB의 상태 검사만으로는 외부 제공자까지 포함한 정확히 한 번 전달을 보장하기 어려워, 제공자의 중복 방지 정책과 함께 다뤄야 합니다.

NotificationService 구현

05 / ROLE & CONTRIBUTION

맡은 역할과 기여

Synapse에서 플랫폼 서비스의 인증·결제·알림·감사 기능을 담당했습니다. 여러 서비스가 공통으로 사용하는 계정과 세션을 관리하고, 결제 이벤트의 중복 처리 방지와 알림 실패 처리를 구현했습니다. 세션 변경과 캐시 갱신의 순서, 외부 이벤트의 재처리 기준을 정리해 공통 플랫폼 기능의 일관성을 유지하도록 구성했습니다.

Flutter 프런트엔드는 팀원들과 함께 작업했습니다. 플랫폼 백엔드에서는 auth·billing·notification·audit 모듈의 책임을 나누고, 인증과 구독 흐름을 다루는 테스트 코드를 구성했습니다.

06 / EVIDENCE & LIMITS

무엇을 확인했고, 무엇이 남았나요?

프로젝트 검증 당시의 실행 기록과 코드에 구성한 테스트 항목을 정리했습니다. 각 항목에 실행 조건과 추가 확인이 필요한 범위를 함께 적었습니다.

인증·결제 E2E 테스트 구성

가입·로그인·MFA·Checkout·웹훅·구독 활성화 시나리오를 다루는 테스트 코드를 구성했습니다.

Testcontainers PostgreSQL과 MockMvc를 사용하고 StripeClient·RefreshTokenService는 모킹합니다. 테스트 코드의 범위이며 현재 환경의 실행 결과는 추가 확인이 필요합니다.

검증 근거 보기

플랫폼 실행 조건과 연동 과제

플랫폼 README에 모듈별 구현 상태, Kafka 활성화 조건, 검증 명령과 남은 연동 과제를 정리했습니다.

진행 상태는 dev 브랜치 기준이며 공개 아카이브는 main 브랜치입니다. 재현할 때는 소스와 실행 조건을 함께 맞춰야 합니다.

검증 근거 보기

Flutter 프로토타입

팀에서 함께 작업한 Flutter 화면과 내비게이션은 mock 데이터와 로그인 바이패스를 사용하는 디자인 프로토타입입니다.

전체 화면의 실제 API 연동과 프런트·백엔드를 연결한 통합 E2E는 후속 검증 범위입니다.

검증 근거 보기

현재 범위와 남은 과제

• Flutter는 mock 기반 프로토타입이며 전체 API 연동과 서비스 간 통합 검증은 남아 있습니다.

• cross-service live E2E와 ReviewCompleted 감사 DLT 조사는 플랫폼의 후속 과제입니다.

• Stripe·FCM·SES 운영 계정의 실제 연동과 장시간 부하 검증은 추가로 필요합니다.

• 플랫폼 README의 진행 상태는 dev 브랜치 기준입니다. 공개 main 아카이브에서 재현할 때 브랜치별 구현 차이를 확인해야 합니다.

다음에 보완할 것

• 실제 프론트·Gateway·서비스를 연결한 가입부터 구독·알림까지의 시나리오를 환경과 커밋을 고정해 기록하기

• 중복 이벤트·제공자 타임아웃·캐시 장애의 복구 결과와 감사 DLT 재처리 절차를 공개 근거로 보강하기

07 / SOURCE MATERIAL

코드와 개발 기록

아래 공개 자료에서 구현 범위와 설계 판단을 확인할 수 있습니다. 링크를 누르면 원문으로 이동합니다.

01 팀 책임 분담

https://github.com/MaoEmong/synapse-team-archive/blob/main/README.md

02 플랫폼 서비스 README

https://github.com/MaoEmong/synapse-team-archive/blob/main/synapse-platform-svc/README.md

03 갱신 토큰 서비스

https://github.com/MaoEmong/synapse-team-archive/blob/main/synapse-platform-svc/src/main/java/com/synapse/platform/auth/service/RefreshTokenService.java

04 결제 서비스

https://github.com/MaoEmong/synapse-team-archive/blob/main/synapse-platform-svc/src/main/java/com/synapse/platform/billing/service/BillingService.java

05 인증·결제 E2E 코드

https://github.com/MaoEmong/synapse-team-archive/blob/main/synapse-platform-svc/src/test/java/com/synapse/platform/e2e/AuthBillingE2ETest.java

06 Flutter 프로토타입 상태

https://github.com/MaoEmong/synapse-team-archive/blob/main/synapse-frontend/README.md